dreaming about `ask`

conor mc bride

August 5, 2025

The current status of this document is 'barely started speculative fiction', concerning the total programming language and proof assistant, `ask`. Version one of `ask` is totally a thing, but I'm plotting a do-over.

1 what's the point?

I designed `ask` to be a problem-directed proof and programming system for absolute beginners. Key concerns include the following.

- Where it does not strain a point, the syntax should resemble that of Haskell. It should lead into learning Haskell.
- The document should read as the minutes of a problem solving interaction. All communication should be within the document. No evaporating information!
- Programming and proof should, as activities, resemble each other as closely as possible. The difference is that proving foregrounds the proposition (the type of the proof), while defining foregrounds the definiendum (the thing with the type).
- Classical logic should be supported no the proof side, but should stick out like a sore thumb.
- Mathematical structure should be distinct from overloading.

You can already tell from the above that I'm unlikely to be happy with any of the systems available off the shelf. Also, I want a thing that's mine to monkey with.

2 the basic picture

An `ask` document is an ordered sequence of declarations and solutions. A solution is a tree which looks like

$$\begin{array}{c} \mathit{problem}\quad \mathit{strategy}\quad \textbf{where} \\ \mathit{subsolution}_1 \\ \vdots \\ \mathit{subsolution}_n \end{array}$$

A *problem* is always introduced by a keyword. Keywords come in two variants: imperative and past participle:

task	imperative	part participle
definition	**define**	**defined**
proof	**prove**	**proven**

The imperative is used for problems with some active task remaining (including the completion of any earlier task on which a solution may depend). The past participle is used in solutions which are hereditarily complete. The keyword is then followed by a problem statement which for definition looks like a 'left-hand side' in functional programming, and for proof is the statement of a proposition.

A *strategy* is always introduced by a keyword (or symbol). The main proof strategy is **by**, invoking some sort of inference rule of introduction. The main programming strategy is to state an equation, so that the whole thing looks like a line of Haskell. Common to both is the **from** strategy, which deploys elimination behaviour. There are other strategies, but we'll get there in good time.

A *subsolution* may begin with **given**, indicating that some proposition holds locally hypothetically, or stating the type of a variable being brought into scope. The latter should be suppressed by default, but permitted. If the user writes x :: without giving the type, `ask` should fill the type in, by return.

3 data declaration problems

The workhorse of `ask` is the inductive data type. A type constructor is an identifier with a capital initial or an infix operator beginning with a colon. Data types offer a choice of value constructors, with the same identifier convention as type constructors — a capital thing is a proper noun, standing only for itself. Haskell notation is supported.

$$\textbf{data}\quad \mathsf{Tree}\ a \quad = \quad \mathsf{Leaf} \quad | \quad \mathsf{Node}\ (\mathsf{Tree}\ a)\ a\ (\mathsf{Tree}\ a)$$

But let us consider possible variations! The first is simply to allow *naming*

$$\textbf{data}\quad xt\ ::\ \mathsf{Tree}\ a \quad = \quad \mathsf{Leaf} \quad | \quad \mathsf{Node}\ (xlt\ ::\ \mathsf{Tree}\ a)\ (x\ ::\ a)\ (xrt\ ::\ \mathsf{Tree}\ a)$$

One motivation is to explain suffix naming conventions for the type. The identifier in the declaration of the type constructor says what we consider typical elements to be called; those in the value constructor name the parts. The longest common prefix of these names (e.g., x) above stands for any prefix, and the remaining suffices can be matched for and applied, so that a tree called *foot*, when split, gets a case for Node *foolt foo foort*.

Note that we should also allow type signatures on parameters/indices:

$$\textbf{data}\quad \mathsf{Tree}\ (a\ ::\ \mathsf{Type}) = \cdots$$

In a minor variation from Haskell, the declaration of an empty type must have an = sign, with nothing thereafter, because it is the innermost *strategy* for data type construction, namely to offer a choice of value constructors. What are other strategies?

Firstly, let us allow the empty strategy.

$$\begin{array}{l}\textbf{data}\quad \mathsf{Tree}\ (a\ ::\ \mathsf{Type})\quad \textbf{where}\\ \qquad \mathsf{Tree}\ a\quad ?\\ \qquad \mathsf{Tree}\ a\quad ?\end{array}$$

We may make as many copies of the problem as we like (zero? probably), meaning that the type is their union. E.g.,

$$\begin{array}{lll}\textbf{data} & \mathsf{Tree}\ (a\ ::\ \mathsf{Type}) & \textbf{where}\\ \quad \textbf{data} & \mathsf{Tree}\ a \quad = & \mathsf{Leaf}\\ \quad \textbf{data} & \mathsf{Tree}\ a \quad = & \mathsf{Node}\ (\mathsf{Tree}\ a)\ a\ (\mathsf{Tree}\ a)\end{array}$$

But now we can consider allowing **from** on indices which happen to be data.

```
data  Vec (n :: Number) (a :: Type)  from n
  data  Vec Z a    =   Nil
  data  Vec (S n) a    =   Cons a (Vec n a)
```

Where I'm going with this is to allow *refinement*. We have enough places we can write **where** clauses that we can indicate proof problems which must be discharged whenever we seek to use the constructor, and which are exposed as **given** when **from** splits data. Here's one I made earlier

```
data  BST l u  where
  data  BST l u    =   Leaf  where
    prove  Le l u
  data  BST l u    =   Node (BST l (Val k)) (k :: Key) (BST (Val k) u)
```

Note that k is used left of its declaration and I don't care.

Oh yeah, and you're allowed to overload constructors. As long as the value constructors for any given type constructor are distinct, you can reuse them (at the potential cost of requiring disambiguation by type signature).

4 structure is not class

A data type like

```
data  S02    =   C0   |   C2 S02 S02
```

can represent an algebraic signature, but how shall we talk about structure?

The usual Haskell way is with type classes, using **newtype** to tell structures apart by forcing the types apart, meanwhile requiring heavy use of overloading to obtain structural benefits. I claim there are too many semigroups about for all of their operations to be called <>. And God help you if you want to assemble a structure like a rig, from two different monoids on the same carrier.

What I want is to recognize and organise the way existing objects already out there in the wild come together in a structure. Dream a dream!

```
structure  Monoid  where
  model  S02
  prove  C2 C0 y = y
  prove  C2 x C0 = x
  prove  C2 (C2 x y) z = C2 x (C2 y z)
```

We might then say

```
structure  addition :: Monoid  where
  model  S02  as  Number  where
    model  C0  as  Z
    model  C2  as  (+)
  prove  Z + y = y   ?
  prove  x + Z = x   ?
  prove  (x + y) + z = x + (y + z)   ?
```